



File System Interface

Ivy Peng
Associate Professor
EECS, KTH



Outline

- **Files**
- File Directory
- Linux File System
- File Sharing and Protection
- Swap Space (Cont.)

File Systems

- File Systems provide a mechanism to access data in storage
- Two main parts:
 - A collection of **files**
 - A **directory structure** that organizes and provides information about all the files

Check different file systems on your system

\$df

Filesystem	1K-blocks	Used	Available	Use%	Mounted on
tmpfs	399636	1912	397724	1%	/run
/dev/vda2	65222220	32453692	29423020	53%	/
tmpfs	1998168	0	1998168	0%	/dev/shm
tmpfs	5120	4	5116	1%	/run/lock
efivarfs	256	26	231	10%	/sys/firmware/efi/efivars
/dev/vda1	524000	5392	518608	2%	/boot/efi
tmpfs	399632	124	399508	1%	/run/user/1000
share	482797568	258813824	223983744	54%	/shared

File Types

- File types can be used to indicate the internal structure of the file.
 - E.g., OS requires that an executable file to have a specific structure to know the location of the first instruction
- UNIX defines all files to be simply streams of bytes
- Programs provided as part of the system - loaders, assemblers, compilers, utility routines, libraries, and so on - are generally defined as files.

file type	usual extension	function
executable	exe, com, bin or none	ready-to-run machine-language program
object	obj, o	compiled, machine language, not linked
source code	c, cc, java, perl, asm	source code in various languages
batch	bat, sh	commands to the command interpreter



The *stdio* Library

- A standard C library that supports common file operations, which are often implemented atop I/O system calls, such as `open()`, `close()`, `read()`, `write()`
- Opening and Closing a File

```
FILE *fopen(const char *filename, const char * mode);  
  
int fclose(FILE *fd);
```



The *stdio* Library

- `putc` writes a character to a file.
- `fputs` writes a string to a file.
- `fprintf` writes a formatted data to a file

```
int putc(int c, FILE *fd)
```

```
int fputs(const char *str, FILE *fd)
```

```
int fprintf(FILE *fd, const char *format, ...)
```

```
#include <stdio.h>
```

```
int main() {
```

```
    FILE *fd;
```

```
    fd = fopen("test.txt", "w");
```

```
    fputs("This is a sample text file.", fd);
```

```
    fclose(fd);
```

```
    return 0;
```

```
}
```

The *stdio* Library

- `getc` reads a character from a file.
- `fgets` reads a string from a file.
- `fscanf` read a formatted data from a file

```
int getc(FILE *fd)
```

```
char *fgets(char *str, int n, FILE *fd)
```

```
int fscanf(FILE *fd, const char *format, ...)
```

```
#include <stdio.h>
```

```
int main(void) {
```

```
    FILE* fd;
```

```
    char ch[100];
```

```
    fd = fopen("myfile.txt", "r");
```

```
    printf("%s", fgets(ch, 50, fd));
```

```
    fclose(fd);
```

```
    return 0;
```

```
}
```

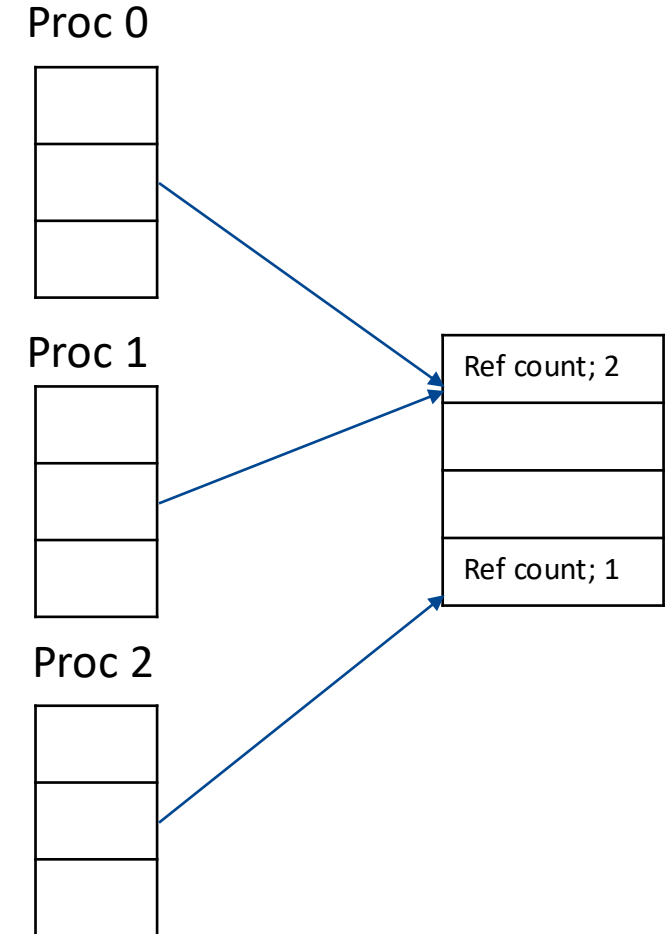


The Open-File Table

- File Operations need to search the file directory to locate the target file
- Constant searching for a file is expensive
- OS keeps an **open-file table** tracking all open files
 - When a file is first used, an `open()` system call is required to insert an entry into the table
 - Each file operation only needs to locate the file in the table

Two-level Open-File Table

- Multiple processes may open the same file
- OS uses two-level tables to track open files:
 - a per-process table: tracks all files opened by a process, e.g., current file pointer, access rights
 - a system-wide table: contains process-independent information, e.g., the location of the file on disk, size
- Each entry in the per-process table points to a system-wide open-file table.

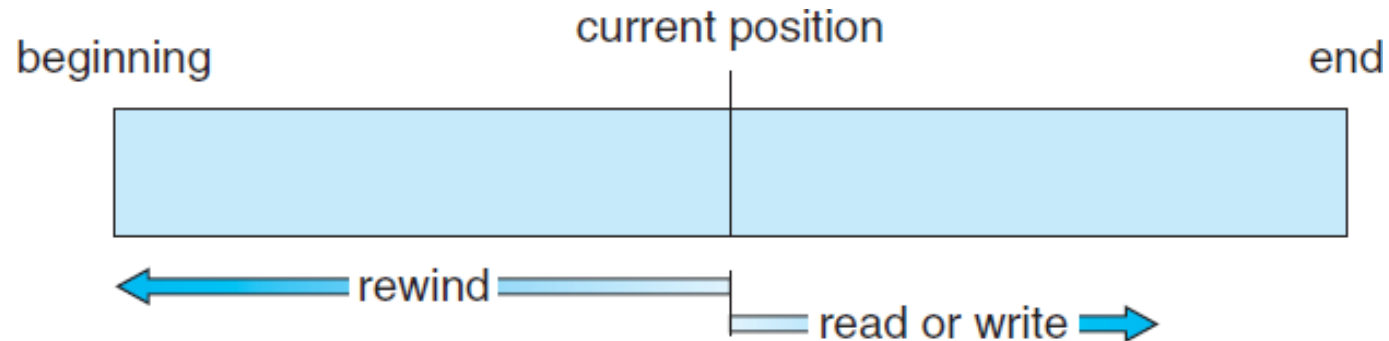


File Locks

- Multiple processes may access the same file at the same time
- File locks allow one process to prevent other processes from gaining access to it
 - Similar to Reader-writer problem
- Two types of locks:
 - A **shared lock** allows several processes to acquire the lock concurrently.
 - An **exclusive lock** only allows one process to acquire the lock

Access Methods - Sequential Access

- Sequential access is based on a tape model of a file.
- Information in the file is processed in order, one record after the other.
- A read operation (`read_next()`): reads the next portion of the file and advances the current position to the next portion.
- A write operation (`write_next()`): appends to the end of the file and advances the current position to the new end



Access Methods - Direct Access

- A file is made up of fixed-length logical records (blocks) that allow programs to read and write records rapidly in no particular order.
- `read(n)` rather than `read_next()`, `write(n)` rather than `write_next()`
- `n` is the block number, often a relative block number that is an index relative to the beginning of the file.
- Databases are often of this type.

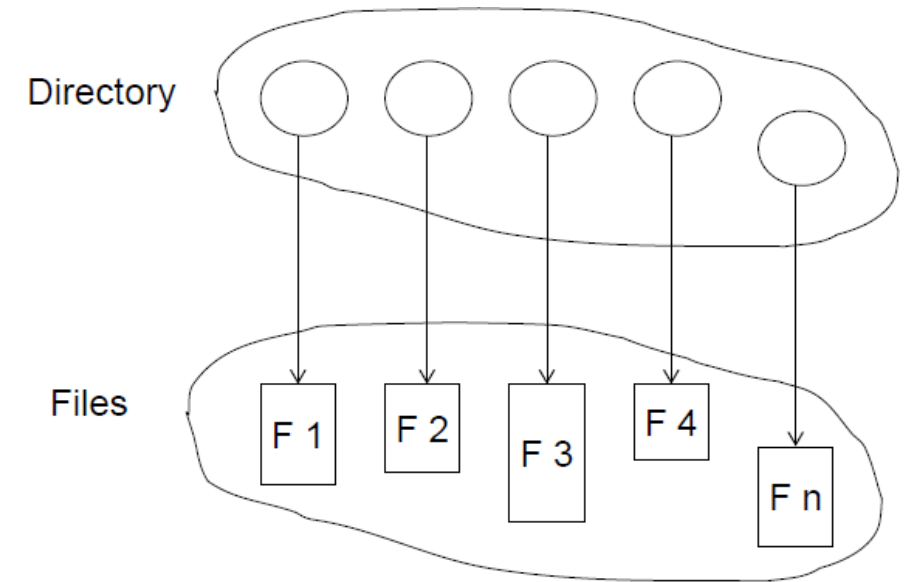


Outline

- Files
- **File Directory**
- Linux File System
- File Sharing and Protection
- Swap Space (Cont.)

File Directory

- Give a list of files, file directory determines how they are organized
- Different file directory structures impact the performance of the following main operations
 - Search for a file
 - Create a file
 - Delete a file
 - List a directory
 - Rename a file
 - Traverse the file system



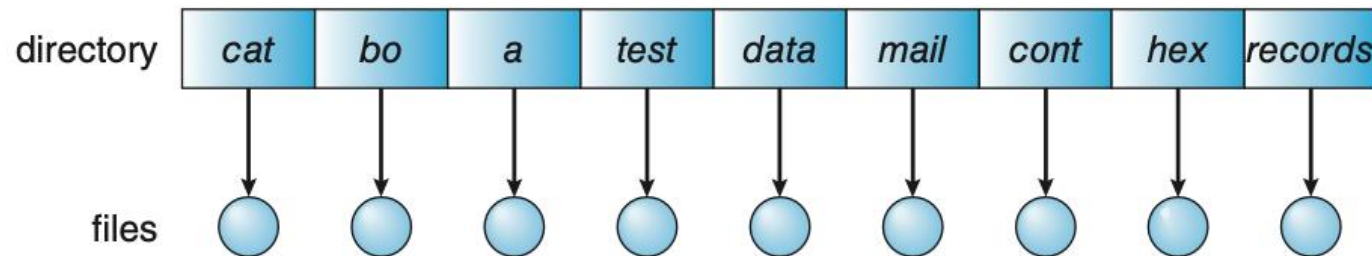


Directory Organization

- The directory itself can be organized in many ways.
 - Single-level directories
 - Two-level directories
 - Tree-level directories
 - Acyclic-graph directories

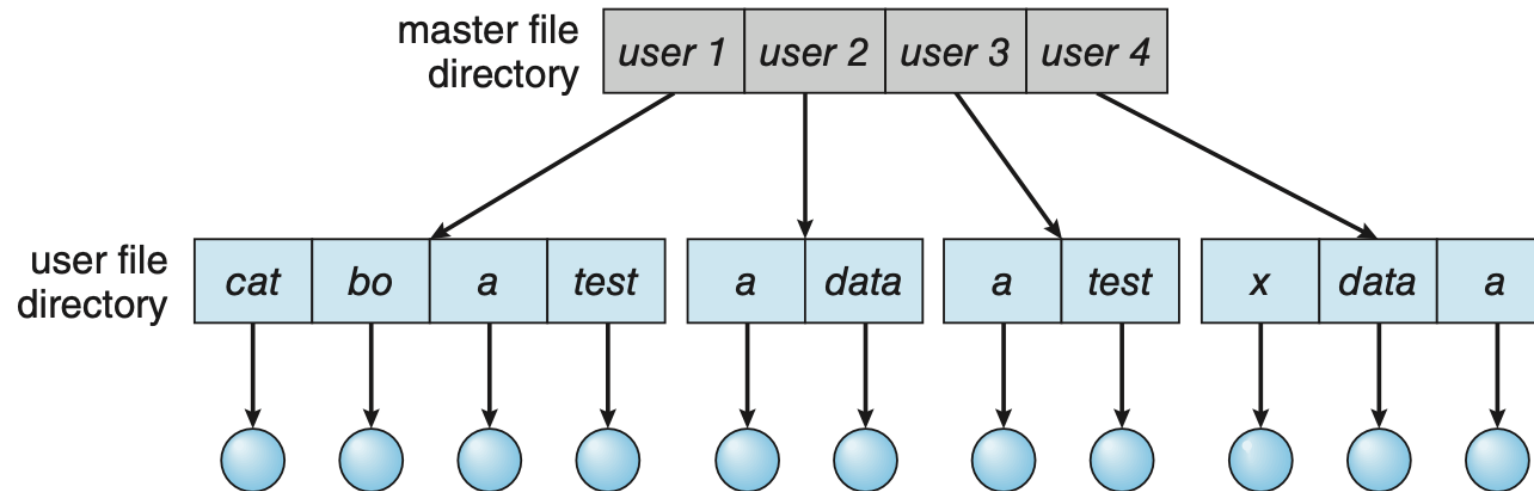
Single-Level Directory

- All files are contained in the same directory
- Simple to implement
- Not scalable when the number of files increases.
- A single user could have a large number of files
- Multiple users on the system may create files of the same name



Two-Level Directory

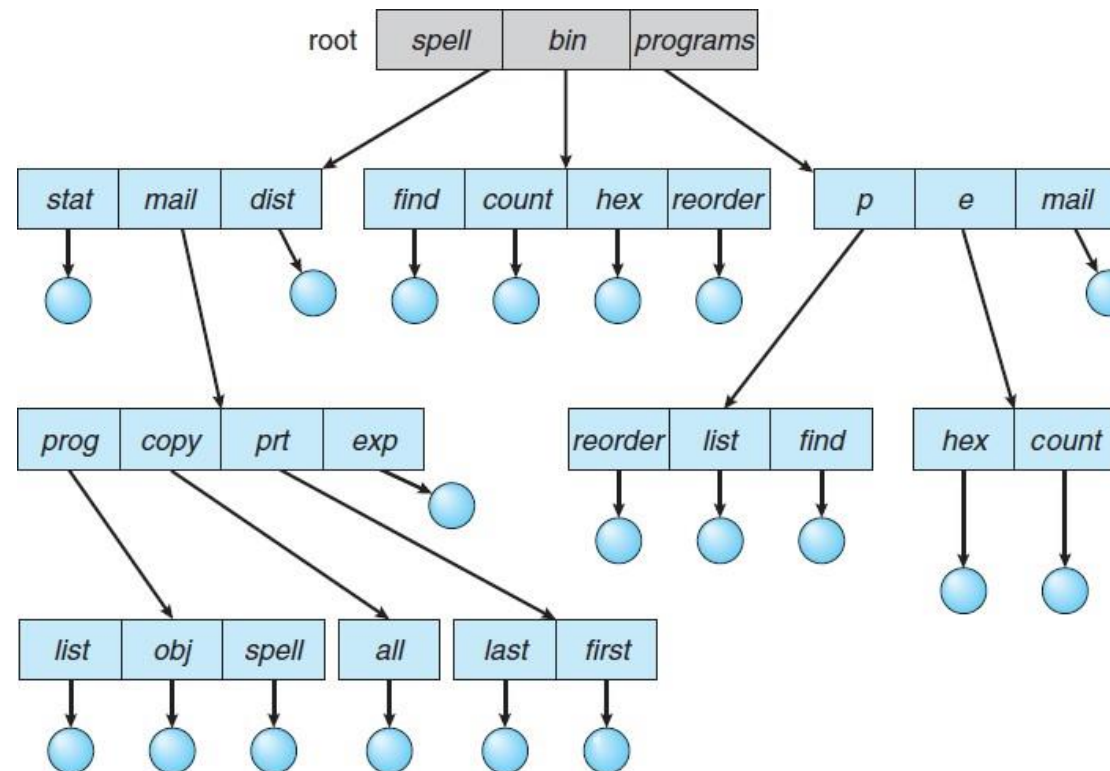
- Create a separate directory for each user. Each user has its own UFD (user file directory)
- A system maintains a system master file directory (MFD)
- Each entry in MFD points to a UFD
- Search is efficient because searching a file is confined into a specific user's directory
- Multiple users on the system may have files of the same name: /usr1/file0 and /usr2/file0



Tree-Structured Directories

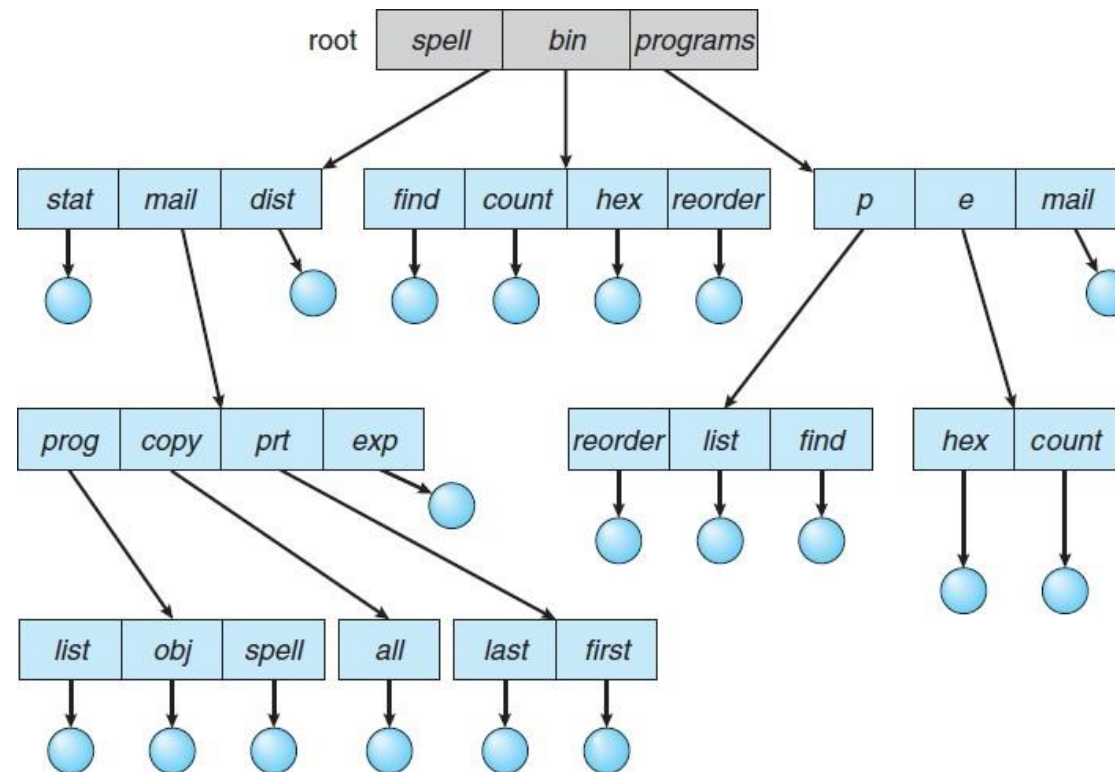
- A generalized two-level directories, allowing each user to further create sub-directories
- Each directory may contain files and/or sub-directories
 - Use 1 bit in each directory entry to indicate whether it is a directory ('d') or file ('-')

-rw-rw-r--	1 pbg	staff	31200	Sep 3 08:30	intro.ps
drwx-----	5 pbg	staff	512	Jul 8 09:33	private/



Tree-Structured Directories

- Current directory (working directory): `cd /spell/mail/prog`
- Two types of path names:
 - **Absolute** path name: a path from the root.
 - **Relative** path name: a path from the current directory. Question: what is the relative path to `/spell/mail/exp` ?



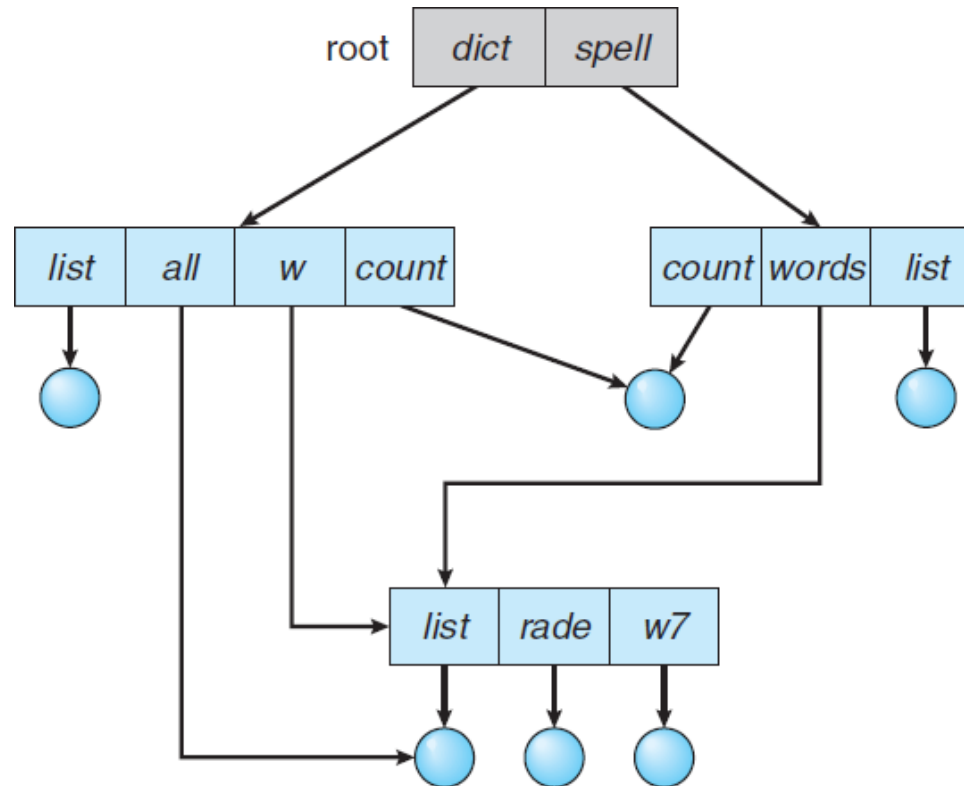
-
- ```

graph TD
 root[root | dict | spell] --> L1[list | all | w | count]
 root --> R1[count | words | list]
 L1 --> C1(())
 L1 --> L2[list | rade | w7]
 L1 --> C2(())
 L2 --> C3(())
 L2 --> C4(())
 L2 --> C5(())
 R1 --> C6(())
 R1 --> C7(())

```

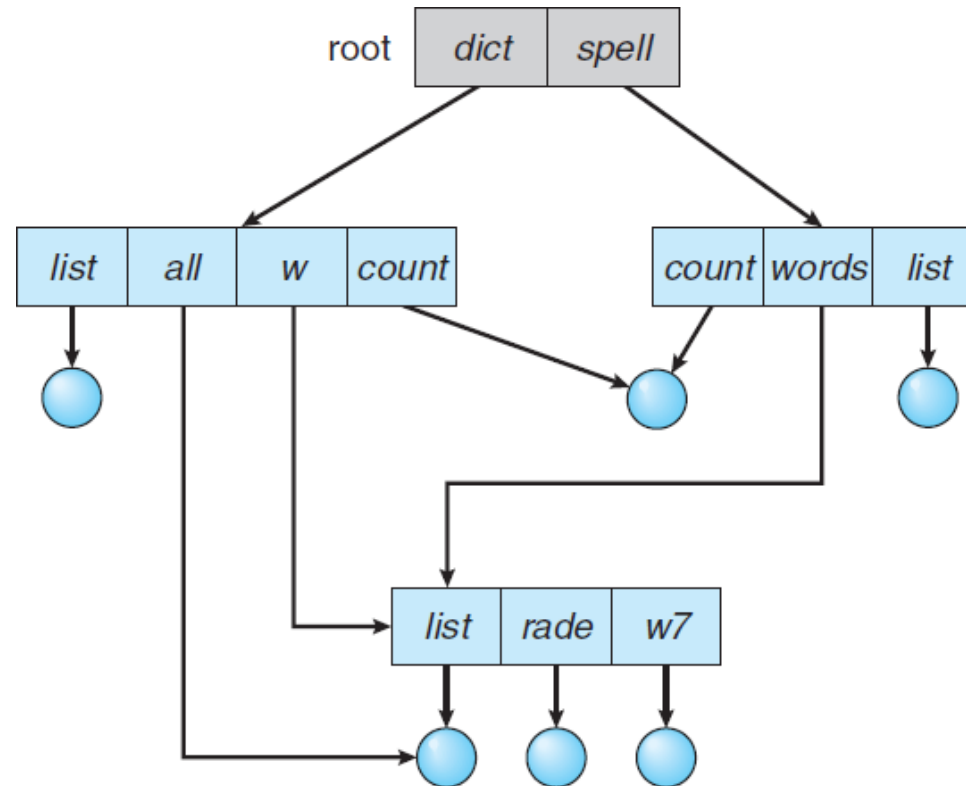
# Acyclic-Graph Directories

- Shared Files can be implemented with Link: another name (pointer) to an existing file.
- Resolve the link: follow pointer to locate the file.



# Acyclic-Graph Directories

- File Deletion can be implemented as:
- Soft (Symbolic) Link: when a file is deleted, keep links (they become dangling pointers), only resolve when an attempt is made.
- Hard (Nonsymbolic) Link: maintain a file-reference list, only remove a file when reference is 0



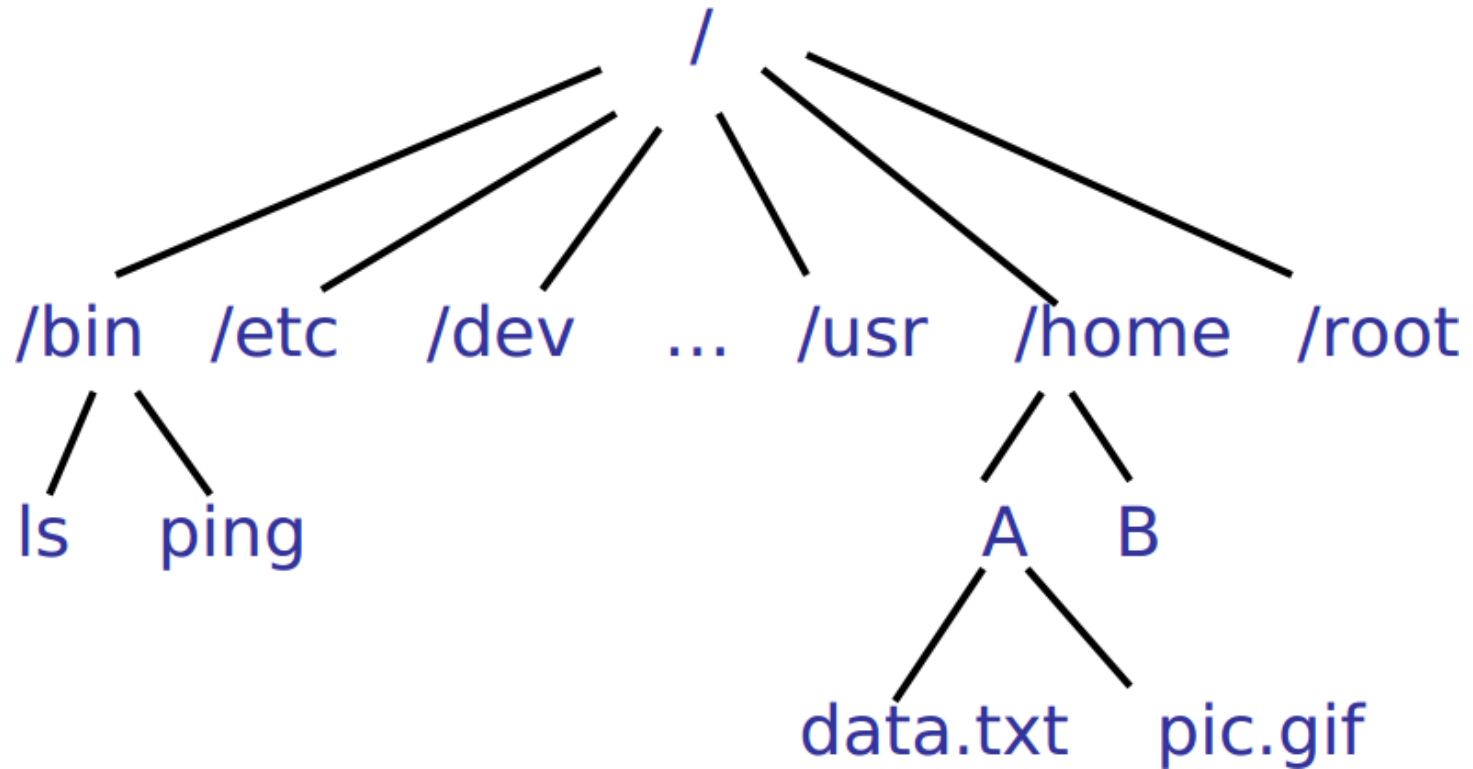


# Outline

- Files
- File Directory
- **Linux File System**
- File Sharing and Protection
- Swap Space (Cont.)

# Linux File System

- From Root at the top



- Hold the most commonly used **user** programs/executables.
  - login
  - Shells (bash, ksh, csh)
  - File manipulation utilities (cp, mv, rm, ln, tar)
  - Editors (ed, vi)
  - File system utilities (dd, df, mount, umount, sync)
  - System utilities (uname, hostname, arch)
  - GNU utilities (gzip, gunzip)

- Hold essential maintenance or **system** programs:
  - fsck, fdisk, mkfs, shutdown, init, ...
- The main difference between the programs stored in /bin and /sbin is that the programs in /sbin are executable only by root



# /etc

- Store the system wide configuration files required by many programs:
- passwd, shadow, fstab, hosts, ...



# **/home and /root**

- The /home directory: the home directories for all users.
- The /root directory: the home directories for root user

- The special files representing hardware are kept in it.
  - /dev/hda1
  - /dev/ttyS0
  - /dev/mouse
  - /dev/fd0
  - /dev/fifo1
  - /dev/loop2



# /tmp and /var

- The /tmp and /var directories: hold temporary files or files with constantly varying content.
- The /tmp directory: files that only need to be used briefly and can afford to be deleted at any time.
- The /var directory: a bit more structured than /tmp.

- Most programs and files directly relating to users of the system are stored.
- It is in some ways a mini version of the / directory.
  - /usr/bin
  - /usr/sbin
  - /usr/spool



# /proc

- It is a virtual file system
- Provided by the kernel
- Provides information about the kernel and processes



# File System Commands

- pwd: where am I?
- cd: changes working directory.
- ls: shows the contents of current directory.
- cat: takes all input and outputs it to a file or other source.
- mkdir: creates a new directory
- rmdir: removes empty directory



# File System Commands

- mv: moves files
- cp: copies files
- rm: removes directory
- gzip/gunzip: to compress and uncompress a file
- tar: to compress and uncompress a file
- e2fsck: check a Linux ext2/ext3/ext4 file system



# File System Commands

- dd: converts and copies a file
- df: reports File System disk space usage
- du: estimates file space usage
- ln: makes links between files
- file: determines file type

# File and Directory Management

- `getcwd()` returns the current working directory.
- `chdir()` changes the current working directory to path
- `mkdir()` creates the directory path
- `rmdir()` removes a directory from the filesystem

```
#include <unistd.h>

char *getcwd(char *buf, size_t size);

int chdir(const char *path);
int mkdir(const char *path, mode_t mode);
int rmdir(const char *path);
```



# File and Directory Management

- `opendir()` creates a directory stream representing.
- `readdir()` returns the next entry in the directory.
- `closedir()` closes the directory stream.

```
#include <sys/types.h>
#include <dirent.h>

DIR *opendir(const char *name) ;;
struct dirent *readdir(DIR *dir);
int closedir(DIR *dir);
```



# Outline

- Files
- File Directory
- Linux File System
- **File Sharing and Protection**
- Swap Space (Cont.)

# File Attributes

File attributes may be changed by Owner of a file/directory:

- Name: only information kept in human-readable form.
- Identifier: unique number identifies file within file system.
- Type: needed for systems that support different types.
- Location: pointer to file location on device.
- Size: current file size.
- Protection: controls who can do reading, writing, executing.
- Time, date, and user identification: data for protection, security, and usage monitoring.
- Information about files are kept in the directory structure.

# File Metadata

- The stat structure: the metadata of a file.
- Defined in <sys/stat.h>.

```
struct stat {
 dev_t st_dev; /* ID of device containing file */
 ino_t st_ino; /* inode number */
 mode_t st_mode; /* permissions */
 nlink_t st_nlink; /* number of hard links */
 uid_t st_uid; /* user ID of owner */
 gid_t st_gid; /* group ID of owner */
 dev_t st_rdev; /* device ID (if special file) */
 off_t st_size; /* total size in bytes */
 blksize_t st_blksize; /* blocksize for filesystem I/O */
 blkcnt_t st_blocks; /* number of blocks allocated */
 time_t st_atime; /* last access time */
 time_t st_mtime; /* last modification time */
 time_t st_ctime; /* last status change time */
};
```



# File Sharing

- Sharing of files on multi-user systems is desirable.
- Sharing may be done through a protection scheme.
  - User IDs identify user
  - **Owner** of a file/directory: the user who can change attributes and grant access and who has the most control over the file.
  - **Group** of a file/directory: a subset of users who can share access to the file.

# Access Lists and Groups

- Three classes of users:
  - Owner: the user who created the file.
  - Group: a set of users who are sharing the file and need similar access.
  - Universe/Public: all other users in the system.
- Each class of user has three modes of access:
  - Read, write, execute (rwx)
- Use of 'chmod' to change the access list of a file:

Question: what access does a public user have?

owner    group    public  
    /      |      \  
chmod 761 game

Owner access rwx: 111 (7)

Group access rwx: 110 (6)

Public access rwx: 001 (1)

# Access Lists and Groups

- use command 'ls' to check the list of file and their info

|            |       |         |       |              |               |
|------------|-------|---------|-------|--------------|---------------|
| -rw-rw-r-- | 1 pbg | staff   | 31200 | Sep 3 08:30  | intro.ps      |
| drwx-----  | 5 pbg | staff   | 512   | Jul 8 09:33  | private/      |
| drwxrwxr-x | 2 pbg | staff   | 512   | Jul 8 09:35  | doc/          |
| drwxrwx--- | 2 jpg | student | 512   | Aug 3 14:13  | student-proj/ |
| -rw-r--r-- | 1 pbg | staff   | 9423  | Feb 24 2012  | program.c     |
| -rwxr-xr-x | 1 pbg | staff   | 20471 | Feb 24 2012  | program       |
| drwx--x--x | 4 tag | faculty | 512   | Jul 31 10:31 | lib/          |
| drwx-----  | 3 pbg | staff   | 1024  | Aug 29 06:52 | mail/         |
| drwxrwxrwx | 3 pbg | staff   | 512   | Jul 8 09:35  | test/         |

Question 1: which are directories?

Question 2: which file(s) are executable by both pbg and staff?

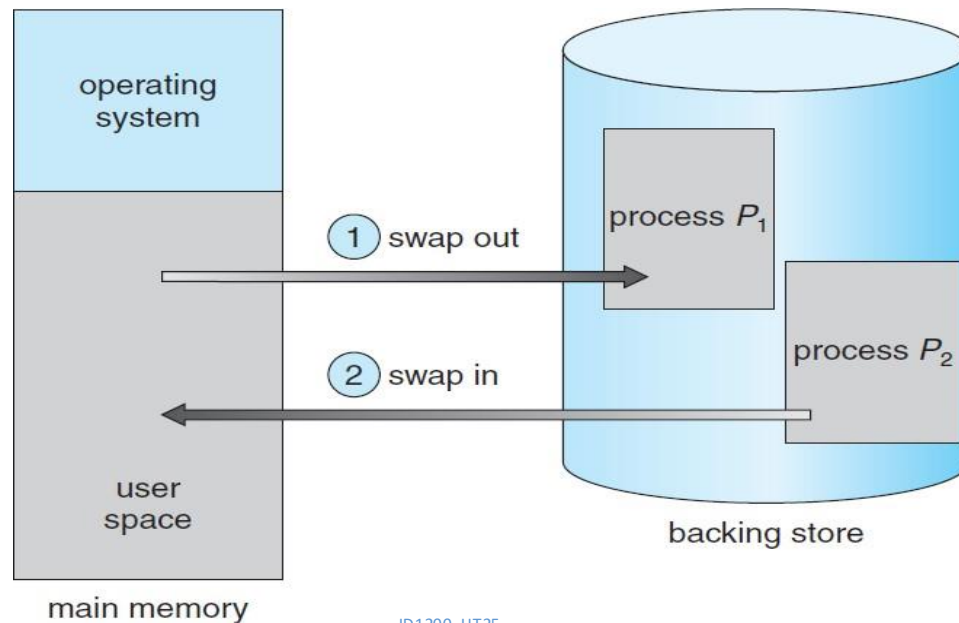


# Outline

- Files
- File Directory
- Linux File System
- File Sharing and Protection
- **Swap Space (Cont.)**

# Recap: Swapping

- Swapping a whole process, as covered in memory management, is rare
  - A process can be swapped temporarily out of memory to disk, and then brought back into memory for continued execution.
- Swapping pages as in virtual memory management is more common



# Swap Space

- Virtual Memory uses storage space to extend main memory
  - As seen in demand paging, when there is no free frame in physical memory, a victim page is swapped out to storage
- OS manages swap space emphasizing on performance optimization as storage can be 1000x slower than DRAM memory
- Two types of Swap Space:
  - A **file** in a normal file system
  - A **raw partition** on the storage

# Optional: Swap Space in Linux

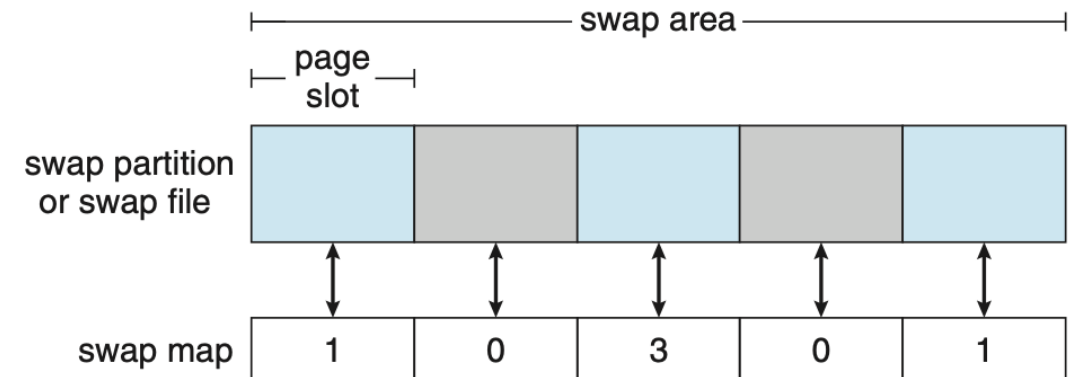
- Only anonymous memory is supported to use swap space
- Supports both types File and Partition swap space
- Linux uses **struct** *swap\_info\_struct* describing a swap space
- Other details:
  - Implemented in `do_swap_page()` under `mm/memory.c`
  - Huge pages are split into 4KB base pages before swapping

```
struct swap_info_struct {
 unsigned int flags;
 kdev_t swap_device;
 spinlock_t sdev_lock;
 struct dentry * swap_file;
 struct vfsmount *swap_vfsmnt;
 unsigned short * swap_map;
 unsigned int lowest_bit;
 unsigned int highest_bit;
 unsigned int cluster_next;
 unsigned int cluster_nr;
 int prio;
 int pages;
 unsigned long max;
 int next;
};
```

struct swap\_info\_struct in Linux

# Optional: Swap Space in Linux

- Use 'Swap Slots' and 'Swap Map' to remember which pages are in swap
  - Array of page sized slots, in 4KB
- The **swap\_map** field in `swap_info_struct` maintains a large array with one entry for every swap entry
- The reference counter on swap map tracks the number of users of this slot





# Summary

- Files: type, access, stdio, open file
- File Directory: tree structure, acyclic graph
- Linux File System: /dev, commands
- File Sharing and Protection: access list and group